



Charte de Nommage

Projet :

**Conception et Développement d'une Plateforme Numérique Communautaire de
Gestion et de Redistribution des Dons Alimentaires – Gasp'Zero.**

année universitaire : 2025/2026

Table De Version

Version	Date	Auteur
V 1.0	24/02/2026	Amrani Liliana

Table des matières

Introduction

Règles générales

2.1. Langue utilisée (anglais)	4
2.2. Style de nommage (camelCase / PascalCase)	4
2.3. Noms descriptifs	4
2.4. Cohérence	4

Nommage par type

3.1. Variables	4
3.2. Fonctions	4
3.3. Classes	4
3.4. Composants (React)	4
3.5. Fichiers	5

Structure du projet

4.1. Dossiers	5
4.2. Sous-dossiers	5

Nommage Backend (Node.js)

5.1. Contrôleurs, Service, Modèles	5
--	---

Nommage Frontend (React Native)

6.1. Écrans, Composants	5
-------------------------------	---

Bonnes pratiques

7.1. Nommage basé sur le rôle	5
7.2. Nommage basé sur l'action	6
7.3. Nommage basé sur le contenu	6

Références

1 Introduction

La charte de nommage définit l'ensemble des règles à respecter pour nommer les variables, fichiers, fonctions et composants dans le projet. Elle vise à garantir une meilleure lisibilité, une cohérence globale du code et une collaboration efficace entre les membres de l'équipe.

2 Règles générales

2.1 Langue utilisée (anglais)

Tous les noms utilisés dans le projet doivent être en anglais. L'utilisation d'une langue unique permet d'assurer une meilleure compréhension du code par tous les développeurs, notamment dans un contexte professionnel international.

2.2 Style de nommage (camelCase / PascalCase)

Le projet adopte deux styles principaux de nommage :

- camelCase : utilisé pour les variables et fonctions (ex : userName, createProject)
- PascalCase : utilisé pour les composants et classes (ex : UserProfile, AuthService)

2.3 Noms descriptifs

Les noms doivent être explicites et refléter clairement leur rôle. Il faut éviter les abréviations ambiguës et privilégier des noms complets permettant de comprendre le code sans commentaires.

2.4 Cohérence

Les conventions de nommage doivent être respectées de manière uniforme dans tout le projet. Une bonne cohérence facilite la maintenance, la lecture et le travail en équipe.

3 Nommage par type

3.1 Variables

Les variables doivent être nommées en camelCase et décrire clairement leur contenu. Exemples : userEmail, donationAmount, projectList.

3.2 Fonctions

Les fonctions doivent être nommées en camelCase et commencer par un verbe indiquant l'action. Exemples : createProject, getUserData, deleteDonation.

3.3 Classes

Les classes doivent être nommées en PascalCase. Elles représentent généralement des entités ou des services. Exemples : User, DonationService, ProjectModel.

3.4 Composants (React)

Les composants React doivent être nommés en PascalCase et représenter une interface utilisateur. Exemples : LoginScreen, DonationCard, UserProfile.

3.5 Fichiers

Les noms de fichiers doivent être descriptifs et correspondre à leur contenu. Exemples :
Frontend : UserProfile.jsx, LoginScreen.jsx
Backend : userController.js, authService.js

4 *Structure du projet*

4.1 Dossiers

Les dossiers doivent être nommés en minuscules et refléter leur rôle. Exemples : controllers, models, services, components

4.2 Sous-dossiers

Les sous-dossiers suivent les mêmes règles et permettent d'organiser le projet de manière claire. Exemple : components/auth, services/payment.

5 *Nommage Backend (Node.js)*

5.1 Contrôleurs

Les contrôleurs doivent être nommés en camelCase avec le suffixe Controller. Exemple : userController.js, projectController.js.

5.2 Services

Les services doivent être nommés avec le suffixe Service. Exemple : authService.js, donationService.js.

5.3 Modèles

Les modèles représentent les données et sont nommés en PascalCase. Exemple : User.js, Donation.js.

6 *Nommage Frontend (React Native)*

6.1 Écrans

Les écrans doivent être nommés en PascalCase avec le suffixe Screen. Exemple : HomeScreen, LoginScreen.

6.2 Composants

Les composants réutilisables doivent être nommés en PascalCase. Exemple : ButtonPrimary, DonationItem.

7 *Bonnes pratiques*

7.1 Nommage basé sur le rôle

Les noms doivent refléter le rôle de l'élément. Exemples : adminUser, donorUser.

7.2 Nommage basé sur l'action

Les fonctions doivent exprimer clairement l'action effectuée. Exemples : createProject, updateProfile.

7.3 Nommage basé sur le contenu

Les noms doivent indiquer le contenu manipulé. Exemples : donationList, userData.

8 Références

Cette charte de nommage s'appuie sur des conventions reconnues dans le domaine du développement logiciel,

Guides et standards de nommage

Google – Google JavaScript Style Guide

<https://google.github.io/styleguide/jsguide.html>

Référence majeure pour les conventions de nommage en JavaScript.

Airbnb – JavaScript Style Guide

<https://github.com/airbnb/javascript>

Très utilisé dans les projets professionnels (frontend & backend).

Documentation officielle

React Documentation

<https://react.dev>

Recommandations sur le nommage des composants (PascalCase).

Node.js Documentation

<https://nodejs.org>

Bonnes pratiques côté backend.

Bonnes pratiques générales

Clean Code – Robert C. Martin (Uncle Bob)

Livre fondamental sur l'importance des noms descriptifs et lisibles.

MDN Web Docs

<https://developer.mozilla.org>

Référence fiable pour les standards JavaScript et web.